MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS

DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATIC

ANDREI ELCISCEV, FAF-243

Laboratory Work No. 1, 2 of Operating Systems

Checked by:
Patricia Reitman, *university assistant*
DISA, FCIM, UTM

Chisinau 2026

Contents

1 Laboratory Work 1: The OS as a Resource Manager **2**

1.1 Objective 2

1.2 Part 1. Files and Directories 2

1.3 Part 2. Processes 3

1.4 Part 3. Memory Management 5

1.5 Part 4. Devices and Storage 5

1.6 Laboratory Work 1: Closing Synthesis 6

2 Laboratory Work 2: Meet the OS You Will Build **8**

2.1 Objective 8

2.2 Part 1. Building and Booting xv6 8

2.3 Part 2. Using xv6 as a Unix System 8

2.4 Part 3. Reading the xv6 Source Code 9

2.5 Part 4. Writing the First xv6 Program 10

2.6 Laboratory Work 2: Conclusion 11

3 General Conclusions **13**

1. Laboratory Work 1: The OS as a Resource Manager

1.1. Objective

The objective of this laboratory work was to observe how a Linux operating system manages four essential resources: files and directories, processes, memory, and devices/storage. The commands were executed in a Linux virtual machine, and the results were analyzed through the terminal and the `/proc` pseudo-filesystem.

1.2. Part 1. Files and Directories

The first part focused on file management. The commands `pwd`, `ls`, `mkdir`, `echo`, `cp`, `mv`, `rm`, and `chmod` were used to navigate the filesystem, create and modify files, and inspect permissions.

Observe Questions and Answers

1) The files created during the exercise were owned by the `root` user and belonged to the `root` group.

2) After applying `chmod 600 note.txt`, the permission string contains ten characters. The first character indicates the file type, while the remaining nine characters represent read, write, and execute permissions for the owner, group, and others. With permission mode 600, the owner has read and write permissions, while the group and other users have no permissions. Therefore, the permission representation is generally `-rw----`.

3) Two directories directly under the root directory are:

   - `/bin`: contains essential executable programs and command binaries.
   - `/dev`: contains device files that provide interfaces to hardware devices and virtual devices.

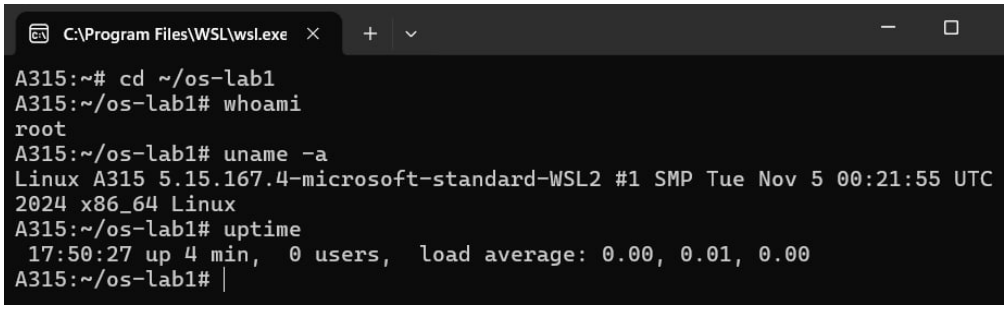


Figure 1.1: Linux user/pc commands

```
A315:~/os-lab1# pwd
/root/os-lab1
A315:~/os-lab1# ls -la /
total 20704
drwxr-xr-x   20 root     root          4096 Sep 28 17:46 .
drwxr-xr-x   20 root     root          4096 Sep 28 17:46 ..
drwxr-xr-x    2 root     root          4096 Apr 18  2025 bin
drwxr-xr-x   11 root     root          3080 Sep 28 17:46 dev
-rwxr-xr-x    1 root     root      18691888 Mar  4  2025 docker-desktop-user
-distro
drwxr-xr-x   20 root     root          4096 Sep 28 17:46 etc
drwxr-xr-x    2 root     root          4096 May 22  2024 home
-rwxrwxrwx    1 root     root       2424984 Mar 19  2025 init
drwxr-xr-x    8 root     root          4096 Apr 18  2025 lib
drwx------    2 root     root         16384 Apr 18  2025 lost+found
drwxr-xr-x    5 root     root          4096 May 22  2024 media
drwxr-xr-x    4 root     root          4096 Apr 18  2025 mnt
drwxr-xr-x    2 root     root          4096 May 22  2024 opt
dr-xr-xr-x  181 root     root             0 Sep 28 17:46 proc
drwx------    4 root     root          4096 Sep 28 13:38 root
drwxr-xr-x    7 root     root           140 Sep 28 17:46 run
drwxr-xr-x    2 root     root          4096 Apr 18  2025 sbin
drwxr-xr-x    2 root     root          4096 May 22  2024 srv
dr-xr-xr-x   11 root     root             0 Sep 28 17:46 sys
drwxrwxrwt    6 root     root          4096 Apr 18  2025 tmp
drwxr-xr-x    7 root     root          4096 Feb 18  2025 usr
drwxr-xr-x   12 root     root          4096 May 22  2024 var
A315:~/os-lab1# ls -la ~
total 20
drwx------    4 root     root          4096 Sep 28 13:38 .
drwxr-xr-x   20 root     root          4096 Sep 28 17:46 ..
-rw-------    1 root     root           968 Sep 28 17:52 .ash_history
drwxr-xr-x    3 root     root          4096 Apr 18  2025 .docker
drwxr-xr-x    3 root     root          4096 Sep 28 14:04 os-lab1
A315:~/os-lab1# cd /etc && ls | head
alpine-release
apk
```

Figure 1.2: File permissions and ownership

1.3. Part 2. Processes

The second part explored process management. The commands `ps`, `top`, `jobs`, `grep`, and `kill` were used to list running processes, start a background process, find its process identifier, and terminate it. The `/proc` filesystem was also used to inspect information about a process.

Observe Questions and Answers

1) The process identifier (PID) observed during the exercise was 41, and it was identified as a sleep job. In a standard Linux system, however, PID 1 is normally assigned to the first userspace initialization process, such as `systemd` or another init process. The exact process associated with a PID should therefore be verified using commands such as `ps -p 1 -f`.

2) Approximately 8 processes were observed on the idle virtual machine at the time of the experiment.

3) The `State:` line in `/proc/<pid>/status` displayed `S`, which means that the process was sleeping or waiting.

```
A315:~/os-lab1# sleep 300 &
A315:~/os-lab1# jobs
[1]+  Running                 sleep 300
A315:~/os-lab1# ps -ef | grep sleep
   38 root      0:00 sleep 300
   40 root      0:00 grep sleep
A315:~/os-lab1# kill 38
A315:~/os-lab1#
[1]+  Terminated              sleep 300
A315:~/os-lab1#
```

Figure 1.3: Process listing and background sleep process

```
C:\Program Files\WSL\wsl.exe
A315:~/os-lab1# sleep 300 &
A315:~/os-lab1# PID=$!
A315:~/os-lab1# ls /proc/$PID/
arch_status      fdinfo          oom_adj         stack
attr             gid_map         oom_score       stat
auxv             io              oom_score_adj   statm
cgroup           limits          pagemap         status
clear_refs       loginuid        personality     syscall
cmdline          map_files       projid_map      task
comm             maps            root            timens_offsets
coredump_filter  mem             sched           timers
cpuset           mountinfo       schedstat       timerslack_ns
cwd              mounts          sessionid       uid_map
environ          mountstats      setgroups       wchan
exe              net             smaps
fd               ns              smaps_rollup
A315:~/os-lab1# cat /proc/$PID/status | head -20
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   42
Ngid:   0
Pid:    42
PPid:   10
TracerPid:      0
Uid:    0       0       0       0
Gid:    0       0       0       0
FDSize: 64
Groups: 0 0 1 2 3 4 6 10 11 20 26 27
NStgid: 42
NSpid:  42
NSpgid: 42
NSsid:  10
VmPeak:     1612 kB
VmSize:     1612 kB
VmLck:         0 kB
VmPin:         0 kB
A315:~/os-lab1# kill $PID
A315:~/os-lab1#
[1]+  Terminated              sleep 300
A315:~/os-lab1#
```

Figure 1.4: Process information from the /proc filesystem

1.4. Part 3. Memory Management

The third part investigated how Linux reports RAM and swap usage. The commands `free -h`, `cat /proc/meminfo`, and `grep VmRSS` were used to examine system memory and the resident memory used by a running process.

Observe Questions and Answers

1) The virtual machine had approximately 7.4 GB of total RAM, of which approximately 7.0 GB was free.

2) Swap is disk space reserved for memory management. When the system needs more RAM or decides to move inactive memory pages away from physical RAM, it can use swap space. The configured swap space was approximately 2.0 GB.

3) A simple `sleep` process used approximately 4 kB of resident memory according to the `VmRSS` value. This result may seem surprising because the program performs almost no visible work. Nevertheless, the process still requires memory for its process structures, execution environment, and other information maintained by the operating system.

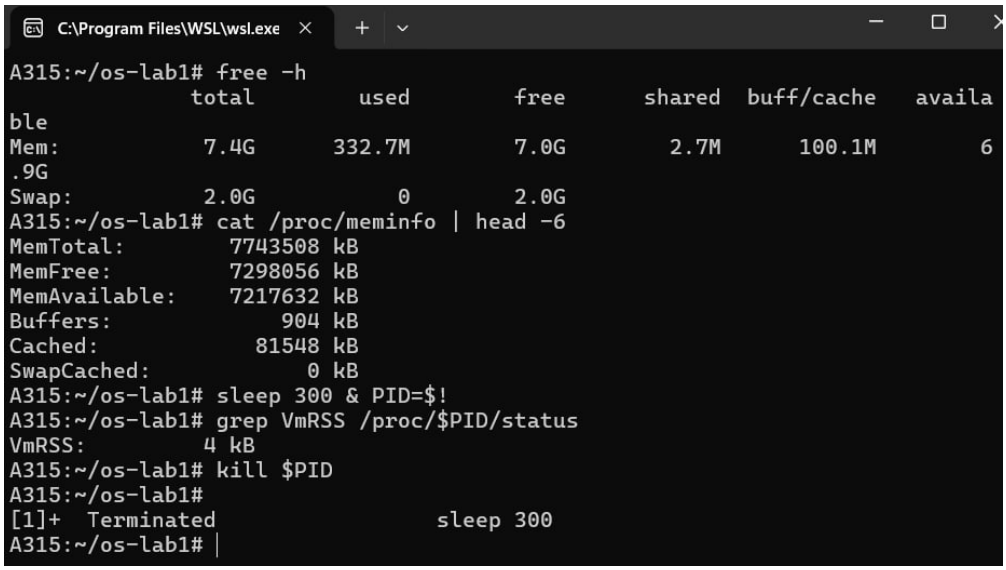


Figure 1.5: RAM, swap, and memory information

1.5. Part 4. Devices and Storage

This part examined storage devices, mounted filesystems, directory sizes, and device files. The commands `df -h`, `lsblk`, `du -sh`, `ls -l /dev`, and `mount` demonstrated how Linux provides a unified interface for storage and devices.

Observe Questions and Answers

1) The root filesystem `/` was mounted on `/dev/sdc`.

2) One entry observed in `/dev` was `console`. It represents a system console interface, related to terminal devices such as `/dev/tty0` and `/dev/tty1`.

3) The expression "everything is a file" means that Linux exposes many resources, including storage devices, terminals, and other hardware interfaces, through file-like objects that can be accessed using common system operations.

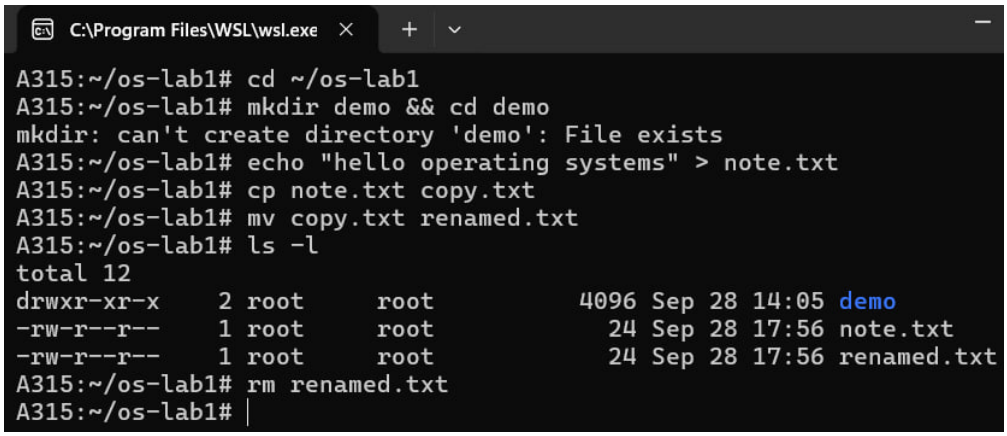


Figure 1.6: CRUD file operations

1.6. Laboratory Work 1: Closing Synthesis

The operating system manages files and directories, processes, memory, and devices/storage. Files and directories were inspected using `ls -la`; processes were observed using `ps aux`; memory was examined using `free -h`; and devices and storage were inspected using `lsblk`. These commands showed that the operating system provides structured interfaces for managing resources and for monitoring their current state.

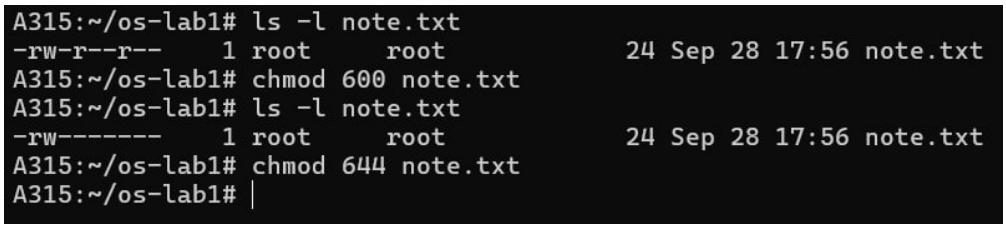


Figure 1.7: Additional Linux observations or final results

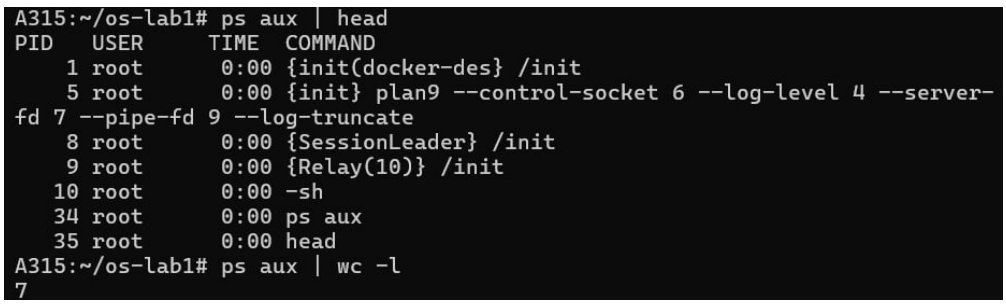


Figure 1.8: Additional Linux command output

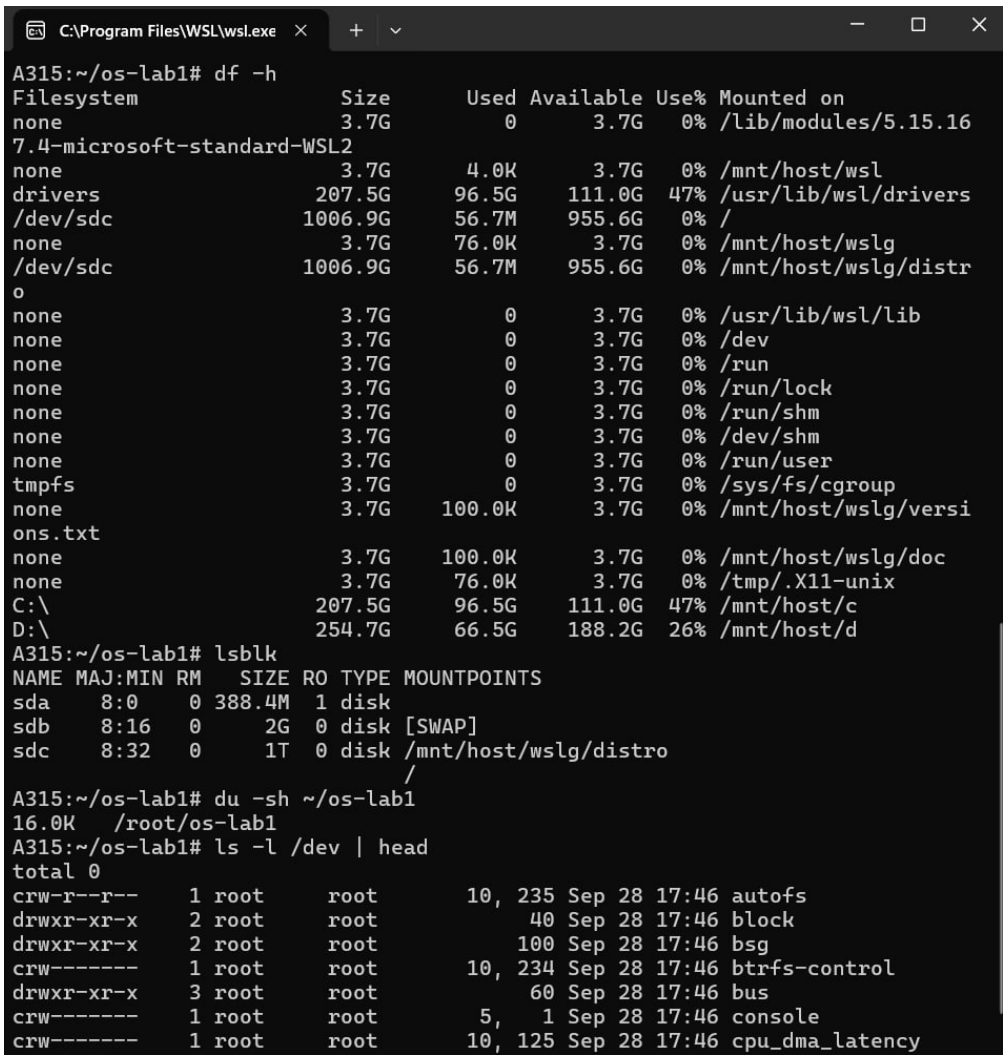


Figure 1.9: Final Linux laboratory evidence

2. Laboratory Work 2: Meet the OS You Will Build

2.1. Objective

The objective of the second laboratory work was to build and boot the xv6 teaching operating system, use its Unix-like shell, inspect its source code, and create a simple user program. This exercise demonstrated the relationship between user programs and kernel functionality, especially the role of system calls.

2.2. Part 1. Building and Booting xv6

The xv6 source code was accessed from the `~/xv6-riscv` directory. The project was compiled and launched using the `make qemu` command. QEMU emulated the hardware environment on which xv6 booted. After the kernel started, the xv6 shell displayed a prompt where commands could be executed.

```
user@user-VirtualBox:~$ cd ~/xv6-riscv 2>/dev/null || \
  git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
Cloning into '/home/user/xv6-riscv'...
remote: Enumerating objects: 8334, done.
remote: Counting objects: 100% (33/33), done.
remote: Compressing objects: 100% (22/22), done.
remote: Total 8334 (delta 19), reused 11 (delta 11), pack-reused 8301 (from 2)
Receiving objects: 100% (8334/8334), 17.67 MiB | 11.53 MiB/s, done.
Resolving deltas: 100% (4583/4583), done.
user@user-VirtualBox:~$ cd ~/xv6-riscv
user@user-VirtualBox:~/xv6-riscv$
```

Figure 2.1: Cloning into github to fetch xv6 system

2.3. Part 2. Using xv6 as a Unix System

The xv6 shell was used to execute built-in programs such as `ls`, `cat`, and `echo`. A pipe was also tested with the command `ls | grep c`.

Observe Questions and Answers

1) Three programs shipped with xv6 are `ls`, `cat`, and `echo`.

2) Two important operating system features are required for a pipe to work:

   - process management, because the commands on the two sides of the pipe execute as processes;
   - inter-process communication (IPC), because the output of one process must be transferred to the input of another process.

3) The xv6 shell is a small Unix-like shell with basic features similar to the Linux shell, including command execution and pipes. However, xv6 provides a more limited environment and fewer utilities than a full Linux distribution.

```
user@user-VirtualBox:~/xv6-riscv$ make qemu
riscv64-linux-gnu-gcc -march=rv64gc -g -ffile-prefix-map=/home/user/xv6-riscv=.
-c -o kernel/entry.o kernel/entry.S
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-p
ointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/user/xv6-riscv=. -march=rv64gc -s
td=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-s
trncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin
-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-st
rchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -
fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-built
in-vprintf -I. -fno-stack-protector -fno-pie -no-pie   -c -o kernel/start.o kern
el/start.c
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-p
ointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/user/xv6-riscv=. -march=rv64gc -s
td=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-s
trncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin
-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-st
rchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -
fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-built
in-vprintf -I. -fno-stack-protector -fno-pie -no-pie   -c -o kernel/console.o ke
rnel/console.c
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-p
ointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/user/xv6-riscv=. -march=rv64gc -s
td=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-s
```

Figure 2.2: Making and building qemu

2.4. Part 3. Reading the xv6 Source Code

The xv6 source tree is divided mainly into the `kernel/` and `user/` directories. The `kernel/` directory contains privileged operating system code, including system-call handlers, process management, memory management, and filesystem functionality. The `user/` directory contains ordinary programs that run on top of the kernel. The files `user/ls.c` and `user/cat.c` were inspected. The `cat` program uses system calls to request services from the kernel.

Observe Questions and Answers

1) The main system calls used by `user/cat.c` are:

   - `read()`: requests that the kernel read data from a file descriptor.
   - `write()`: requests that the kernel write data to a file descriptor.
   - `exit()`: requests that the kernel terminate the current process.

2) The `sys_read` implementation is located in `kernel/sysfile.c`, in the function whose declaration begins with `uint64 sys_read(void)`. The exact line number may differ depending on the xv6 version used.

3) The `kernel/` directory contains privileged operating system code that manages hardware and system resources, while the `user/` directory contains ordinary programs that request services from the kernel through system calls.

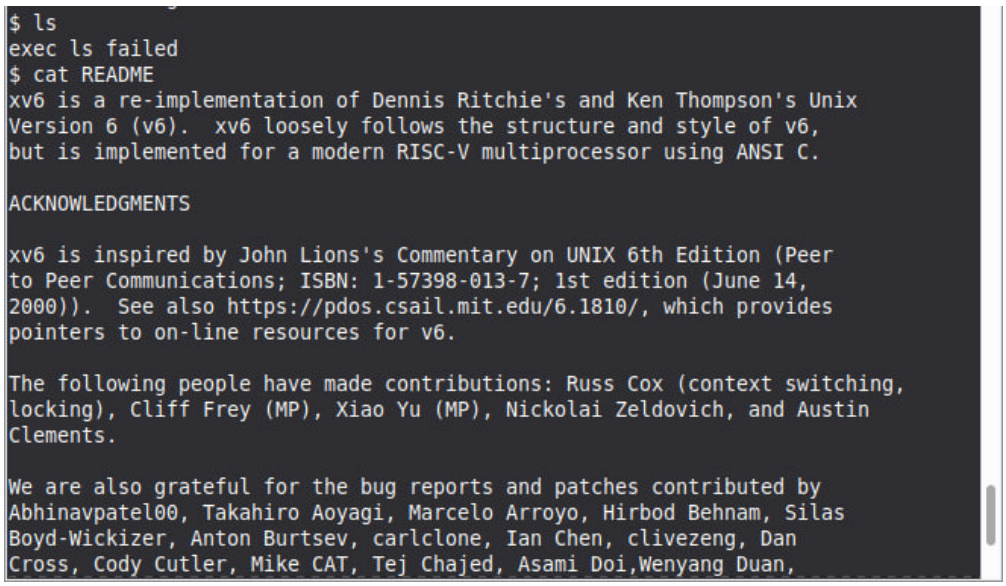


Figure 2.3: Displaying README

2.5. Part 4. Writing the First xv6 Program

The first custom xv6 program was a `sleep` utility. Its purpose was to pause execution for a specified number of clock ticks by calling the existing `sleep()` system call. The program checks whether exactly one command-line argument is provided. If the number of arguments is incorrect, it prints a usage message and exits with an error code. Otherwise, it converts the argument to an integer, calls `sleep()`, and terminates successfully.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }

    sleep(atoi(argv[1]));
    exit(0);
}
```

The program was added to the `UPROGS` list in the Makefile. After rebuilding xv6, the program was tested with `sleep 10`. The incorrect invocation `sleep` was also tested to verify that the usage message was displayed.

```
user@user-VirtualBox:~/xv6-riscv$ sed -n "1,40p" user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

  // Return blank-padded name.
  if (strlen(p) >= DIRSIZ)
    return p;
  memmove(buf, p, strlen(p));
```

Figure 2.4: Listing program contents

```
user@user-VirtualBox:~/xv6-riscv$ sleep 10
user@user-VirtualBox:~/xv6-riscv$ sleep 10
```

Figure 2.5: Testing the sleep program inside xv6

2.6. Laboratory Work 2: Conclusion

This laboratory work demonstrated that xv6 is a small but functional Unix-like operating system. By building and booting it with QEMU, the relationship between the kernel, user programs, and system calls became more concrete. Reading the source code showed that user programs do not directly manage hardware; instead, they request services from privileged kernel code. The custom `sleep` program demonstrated the complete path from a user-level command to a system call implemented by the kernel.

```
user@user-VirtualBox:~/xv6-riscv$ nano Makefile
user@user-VirtualBox:~/xv6-riscv$ nano user/sleep.c
user@user-VirtualBox:~/xv6-riscv$ ^C
user@user-VirtualBox:~/xv6-riscv$ grep -n "sleep" user/user.h
user@user-VirtualBox:~/xv6-riscv$ nano user/user.h
user@user-VirtualBox:~/xv6-riscv$ make clean
make qemu
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
        user/usys.S \
user/_cat user/_echo user/_sleep user/_forktest user/_grep user/_init user/_kill
 user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests
user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan us
er/_sync
riscv64-linux-gnu-gcc -march=rv64gc -g -ffile-prefix-map=/home/user/xv6-riscv=.
-c -o kernel/entry.o kernel/entry.S
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-p
ointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/user/xv6-riscv=. -march=rv64gc -s
td=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-s
trncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin
```

Figure 2.6: Rebuilding QEMU emulator

```
user@user-VirtualBox:~/xv6-riscv$ nano user/sleep.c

user@user-VirtualBox:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1]));
  exit(0);
}
```

Figure 2.7: Final xv6 result or verification

3. General Conclusions

The two laboratory works provided complementary perspectives on operating systems. In Laboratory Work 1, Linux was examined from the outside through terminal commands that revealed the management of files, processes, memory, and devices. The `/proc` filesystem and device-file interface demonstrated how the operating system exposes information about running processes and hardware resources.

In Laboratory Work 2, xv6 provided an opportunity to examine the operating system from the inside. Its source code showed the separation between privileged kernel functionality and ordinary user programs. The implementation and testing of the `sleep` program illustrated how a user application invokes a system call and receives a service from the kernel.

Overall, the exercises demonstrated that an operating system acts as an intermediary between applications and hardware. It manages resources, isolates processes, provides system-call interfaces, and creates a consistent environment in which programs can execute. Linux showed these concepts in a practical, fully developed system, while xv6 made the same concepts easier to understand because its implementation is small and accessible.

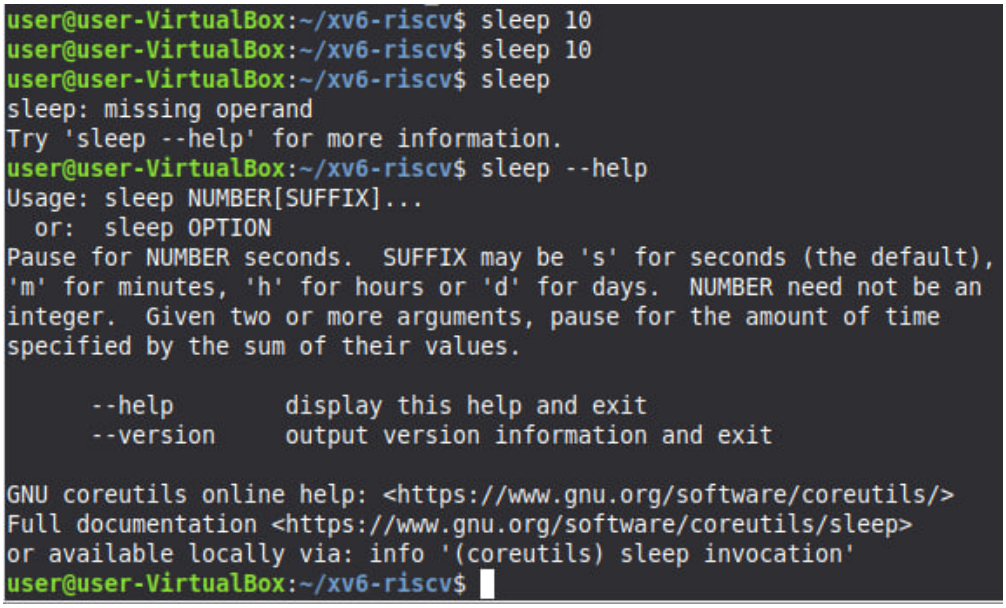


Figure 3.1: Final summary screenshot or selected laboratory evidence